

Memoria

-

Ejercicio React



Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Ruben Cameo Saez

1. Introducción

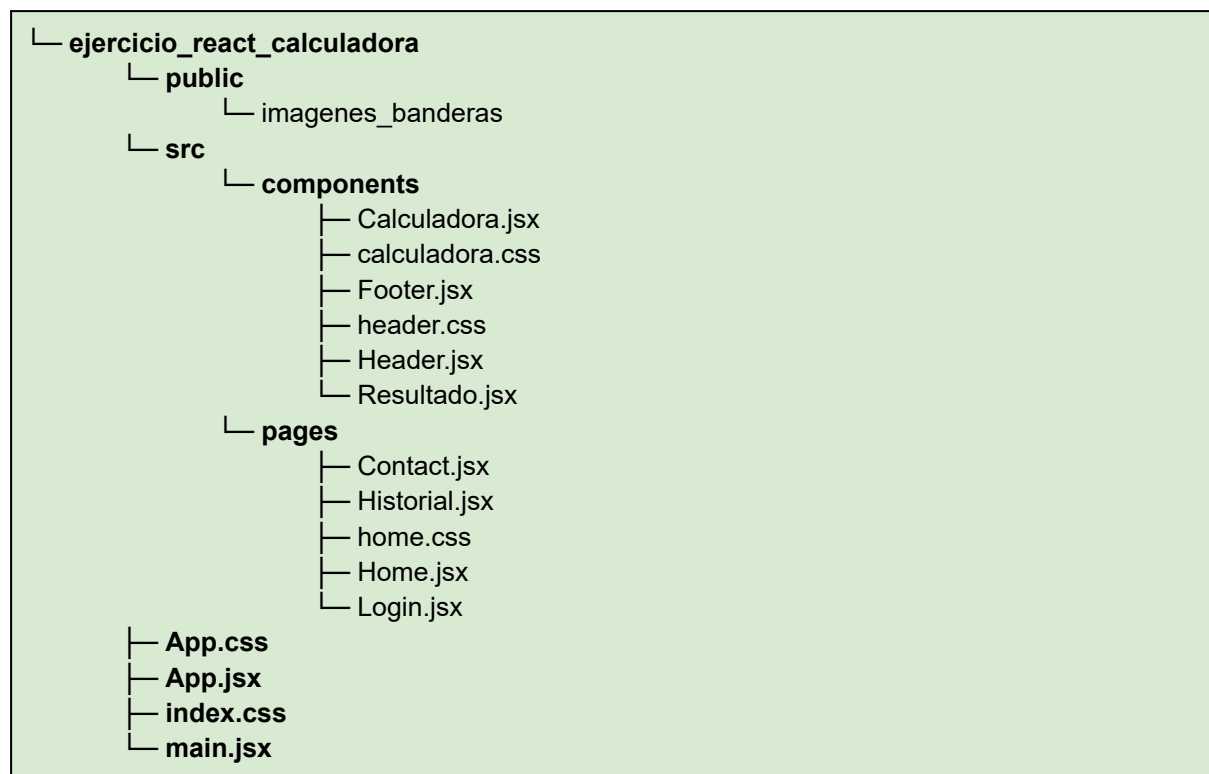
El objetivo de este ejercicio es aplicar y consolidar los conceptos fundamentales de React trabajados en clase, tales como el uso de componentes, el manejo de eventos y la gestión de estados.

Para ello, se ha desarrollado una aplicación que cumple con los siguientes requisitos:

- Creación de dos campos numéricos para los valores A y B.
- Implementación de cuatro botones para las operaciones básicas (+, -, ×, ÷).
- Crear un componente resultado para mostrar la operación pulsada aplicada a los valores A y B.
- Diferenciar de algún modo el botón de operación activo en cada momento.

2. Desarrollo y funcionalidad

Con el objetivo de cumplir los requisitos del ejercicio, se ha definido la siguiente estructura de distribución:



Como se observa en el esquema, el punto de entrada principal es `App.jsx`. Este componente actúa como contenedor global, albergando la estructura base compuesta por el `Header`, el cuerpo principal (`Home`) y el `Footer`.

Dentro de la página principal (`Home.jsx`), se han integrado el componente clave solicitado:

- **Calculadora.jsx:** Es el núcleo de la aplicación, donde reside la lógica de los estados para los valores A y B, así como los botones de operación. Además, este componente invoca al componente `Resultado`.

La vista general de la aplicación es la que se indica a continuación:

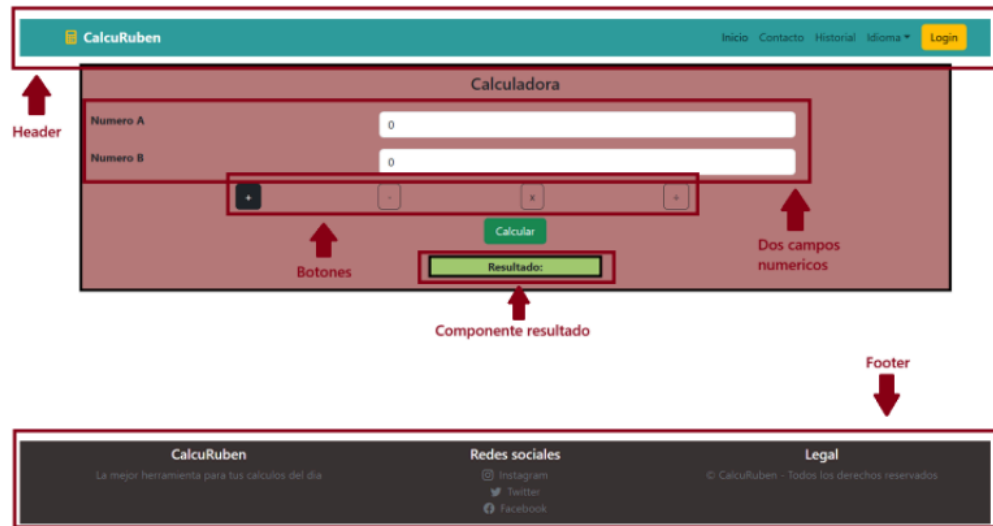


Fig 1. Interfaz de usuario final con la indicación de los componentes funcionales

2.1 App.jsx

Este archivo actúa como el componente raíz y el punto de entrada de la aplicación.

Aspectos clave de la implementación:

- Se emplea una estructura que envuelve el contenido entre un Header y un Footer. Gracias a las clases de Bootstrap (d-flex flex-column min-vh-100), se garantiza que el pie de página se mantenga siempre en la parte inferior de la pantalla, independientemente de la cantidad de contenido.
- Se implementa la librería react-router-dom mediante los componentes <Routes> y <Route>.
- Se ha definido una etiqueta main con la clase flex-grow-1. Esto es fundamental para el diseño, ya que obliga al cuerpo de la aplicación (donde se renderiza el componente Home) a ocupar todo el espacio disponible entre la cabecera y el pie de página.
- Persistencia con localStorage: Se utilizan dos useEffect para gestionar la memoria del navegador. El primero de ellos se ejecuta únicamente al montar el componente para tomar del localStorage el apartado de historial. Como los datos en el navegador se almacenan siempre como texto, se utiliza JSON.parse(), que permite convertirlos de nuevo en un array. El segundo de los useEffect tiene una dependencia con [historial]. De esta forma, cada vez que el usuario realiza un cálculo y el estado del historial cambia, la aplicación ejecuta automáticamente el localStorage.setItem.

- Se emplea una constante denominada `textos` estructurada en un objeto con dos claves principales: `es` (español) y `en` (inglés), que contienen todas las cadenas de texto de la interfaz. Para que el cambio de idioma sea reactivo, se utiliza el estado `idioma` como índice dinámico. De esta forma, al pasar la prop `textos={textos[idioma]}` a los componentes hijos, React detecta automáticamente qué sección del diccionario debe renderizar en cada momento.

2.2 Home.jsx

Este componente va a actuar como la página principal de la aplicación. Su función es estructural y organizativa, sirviendo de puente entre el `App.jsx` y el componente funcional, llamado `Calculadora.jsx`.

Aspectos clave de la implementación:

- Utilizamos un componente `<Container>` con clases de tipo `Flexbox` (`d-flex`, `align-items-center`, `justify-content-center`) para asegurar que la calculadora aparezca perfectamente centrada tanto vertical como horizontalmente en la pantalla.
- He separado la parte visual de la lógica matemática. El archivo `Home.jsx` solo se encarga de mostrar la estructura de la página, mientras que toda la programación de los cálculos y los botones se encuentra dentro del componente `<Calculadora />`.
- Dentro del componente de `Calculadora`, recibimos el idioma y los textos desde el componente superior. Estos datos se pasan hacia los niveles inferiores (como el componente `Resultado`) utilizando las props. De esta manera, conseguimos que toda la calculadora se adapte al idioma seleccionado (español o inglés)

2.3. Calculadora.jsx

En este componente es donde he implementado la lógica principal del ejercicio. En él he gestionado cuatro estados mediante el `useState`: los dos números de entrada (`numA`, `numB`), la operación matemática elegida y el resultado final.

Aspectos clave de la implementación:

- Para los campos de entrada, he usado el evento `onChange`. Es importante destacar que he envuelto el valor en `Number()` para asegurar que React realice operaciones matemáticas reales y no concatene los valores como si fueran texto.
- He creado una función llamada `calcular` que se activa al pulsar el botón correspondiente. Su tarea es recoger los valores almacenados en los estados `numA` y `numB`, y aplicar la operación matemática seleccionada. Una vez obtenido el valor, se utiliza la función `setResultado` para actualizar el estado y que el nuevo dato se refleje en la pantalla. Finalmente, se llama a la función `agregarAlHistorial`, enviando los números y el resultado hacia el componente principal para que la operación quede registrada en el historial de la aplicación.

- He creado la función `marcadosignoelegido` para cumplir con el requisito de diferenciar el botón activo. Esta función compara el estado de operación con el símbolo de cada botón. Si coinciden, devuelve la variante de Bootstrap dark (botón relleno), y si no, devuelve `outline-dark` (solo el borde).
- Se establece de la siguiente forma:
 - La estructura que he utilizado se define mediante un `div` principal con formato `d-flex flex-column` para que todos los elementos queden bien centrados. Después, he implementado otro `div` que hace de caja para la propia calculadora, donde he puesto el título con un `h4`.
 - Dentro de esa caja, he organizado el contenido con Containers. Primero, he puesto un contenedor con filas (`Row`) y columnas (`Col`) para colocar las etiquetas (`labels`) y los cuadros de entrada (`inputs`) de los números A y B. De esta forma, quedan bien alineados uno debajo del otro.
 - Luego, he añadido otro Container específico para los cuatro botones de las operaciones. Aquí he usado una fila con cuatro columnas, una para cada signo (+, -, x, /). En cada una de estas columnas he metido un botón que tiene un evento `onClick`. Al pulsarlo, se activa la función `setOperacion` para que el sistema sepa qué signo se ha elegido y lo marque como activo.
 - Finalmente, he incluido un último Container abajo del todo donde he colocado tanto el botón de calcular como el componente de Resultado, para que el usuario vea la solución justo después de pulsar el botón.

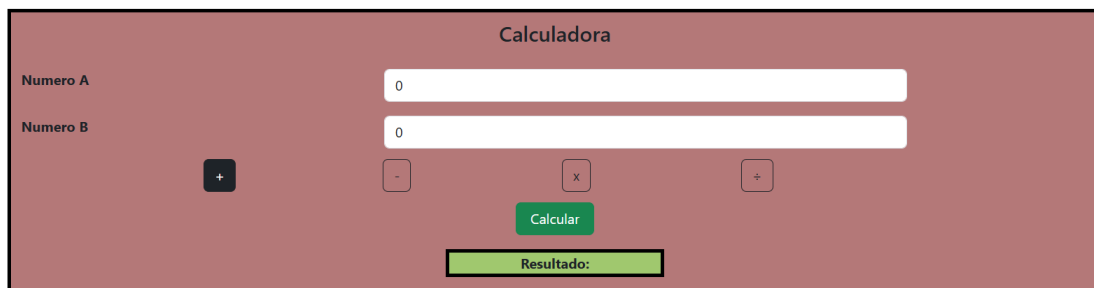


Fig 2. Vista de Calculadora

2.4. Resultado.jsx

Este componente es el encargado de mostrar el número final del cálculo en la pantalla. Su función es simplemente recibir el dato que viene de la calculadora y pintarlo de una forma que quede clara y visual para el usuario.

Aspectos clave de la implementación

- He configurado el componente para que reciba `props.resultado_obtenido`. Esto permite que, en cuanto la calculadora saca un resultado, este valor se renderice automáticamente en este apartado sin tener que recargar nada.

- Para que no fuera un texto simple, he usado un div con clases de Bootstrap (d-flex y align-items-center) para que el resultado siempre aparezca centrado. Además, le he puesto estilos propios
- Al tenerlo como un archivo separado (export default), el código queda mucho más organizado. Si en el futuro quisiera mostrar el resultado en otra parte de la web, solo tendría que llamar a este componente y pasarle el valor que quiera enseñar.

2.5. Header.jsx

Este componente es la barra de navegación superior que aparece en todas las páginas de la web.

Aspectos clave de la implementación:

- He configurado los enlaces con Link para que el usuario pueda moverse por las secciones (Inicio, Contacto, etc.) de forma inmediata.
- El estilo que he empleado ha sido el de poner primero el <Navbar> y posteriormente un <Container> para englobar los diferentes apartados internos.
 - En primer lugar tenemos el <Navbar.Brand> con el que ponemos el título de la web ("CalcuRuben") y un icono de una calculadora para que quede más visual.
 - Posteriormente he puesto el <Navbar.Toggle> para que, cuando se vea desde un móvil, aparezca el típico botón de menú desplegable.
 - Navegación: Tras ello se coloca el <Navbar.Collapse> para que dentro se coloquen los apartados por los que se puede navegar (Inicio, Contacto e Historial). He configurado estos enlaces con Link para que el cambio entre páginas sea instantáneo.
 - También he incluido un desplegable con las banderas de España e Inglaterra. He programado una pequeña lógica con un if para que el idioma que esté seleccionado en ese momento aparezca con un fondo gris, así el usuario sabe cuál tiene activado.
 - Por último, he añadido un botón de Login en color amarillo para que resalte sobre el fondo turquesa de la barra y sea lo primero que se vea al entrar.



Fig 3. Vista del Header

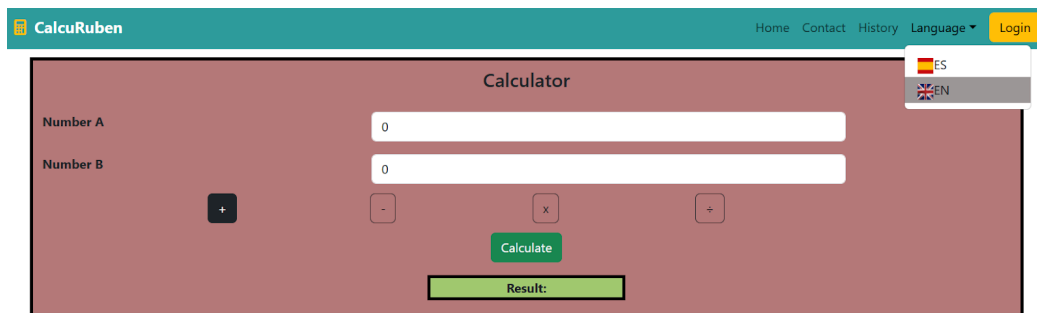


Fig 4. Visualización de selección de idiomas

2.6. Footer.jsx

Este componente es el pie de página de la aplicación.

Aspectos clave de la implementación:

- Estructura completa empleada:
 - El formato que he empleado ha sido el de poner inicialmente un div para englobar todo el elemento y así definir el color de fondo oscuro.
 - Después, dentro de un Container, he utilizado una fila (Row) con tres columnas (Col) para que aparezcan tres apartados diferentes, que es lo que suele ser más típico en los pies de página.
 - Para cada una de estas columnas, he diferenciado entre los tamaños xs y md para que el contenido se adapte bien a cualquier pantalla (móvil o PC). Además, he usado la clase text-center para que todo el texto aparezca centrado.
 - El primer apartado muestra un breve texto informativo. El segundo apartado lo he implementado para indicar las redes sociales, usando iconos para que sea más visual. El tercer apartado es donde he puesto el aviso legal y los derechos reservados.
- Decir que los textos los hemos empleado con el props.textos para que se adapte al idioma elegido

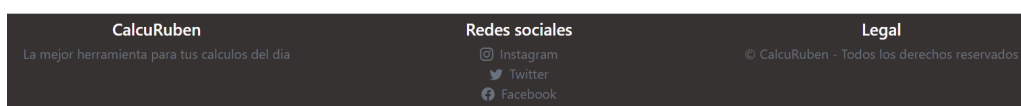


Fig 5. Vista de Footer

2.7. Historial.jsx

Este componente se encarga de mostrar el registro de todas las operaciones que se han realizado.

Aspectos clave de la implementación:

- Para que el historial no fuera una lista simple de texto, he utilizado el componente `<Card>` de Bootstrap. Le he puesto un encabezado (`Card.Header`) con un color y un cuerpo (`Card.Body`). También he añadido la clase `shadow-sm` para darle un poco de sombra y que quede más profesional.
- Esta es la parte más importante del código. Como el historial es un array (una lista de datos) que recibo por props, utilizo la función `.map()`. Lo que hace es recorrer cada operación guardada y crear automáticamente un elemento de lista (`<li>`) para cada una.
- Dentro de cada línea de la lista, pinto los valores de la operación: el primer número, el signo, el segundo número y el resultado final.
- Al usar el index como clave (`key={index}`), le ayudo a React a saber exactamente qué elemento de la lista es cada uno, evitando errores cuando la lista va creciendo.



Fig 6. Vista de Historial

2.8. Contact.jsx

Este componente es la página de contacto donde los usuarios pueden enviar sus dudas o comentarios. Resaltar que no hace funcionalidad como tal, es simplemente para rellenar la aplicación React que se está creando.

Aspectos clave de la implementación:

- He utilizado una <Card> con un borde negro de 2px para que el formulario resalte. Dentro he puesto campos para el nombre, el email y un cuadro de texto más grande (as="textarea") para los comentarios.
- He usado clases como justify-content-center para que el cuadro de contacto aparezca centrado y fw-bold para que los títulos de los campos se vean en negrita.
- Como en el resto de la App, todos los textos (el eslogan, las etiquetas y el botón) cambian de idioma automáticamente gracias a las props.textos.

The screenshot shows a contact form within a teal header bar. The header bar contains the CalcuRuben logo and navigation links: Inicio, Contacto, Historial, Idioma, and a Login button. The form itself is a white card with a black border. It has a title '¡Ponte en contacto con nosotros!' and three input fields: 'Nombre' (with a placeholder 'Mete el nombre identificativo sin puntos ni guiones'), 'Email', and 'Comentarios' (a larger text area with a placeholder 'En que podemos ayudarte'). A 'Enviar mensaje' button is at the bottom of the form. The footer is a dark grey bar with three sections: 'CalcuRuben' (with the tagline 'La mejor herramienta para tus calculos del día'), 'Redes sociales' (with icons for Instagram, Twitter, and Facebook), and 'Legal' (with the text '© CalcuRuben - Todos los derechos reservados').

Fig 7. Vista de área de contacto

3. Implementación para móviles

Para garantizar una experiencia adecuada de usuario en dispositivos móviles, se ha seguido una estrategia de diseño responsivo. El objetivo principal ha sido adaptar los elementos de la interfaz para que mantengan su usabilidad sin importar el tamaño de la pantalla.

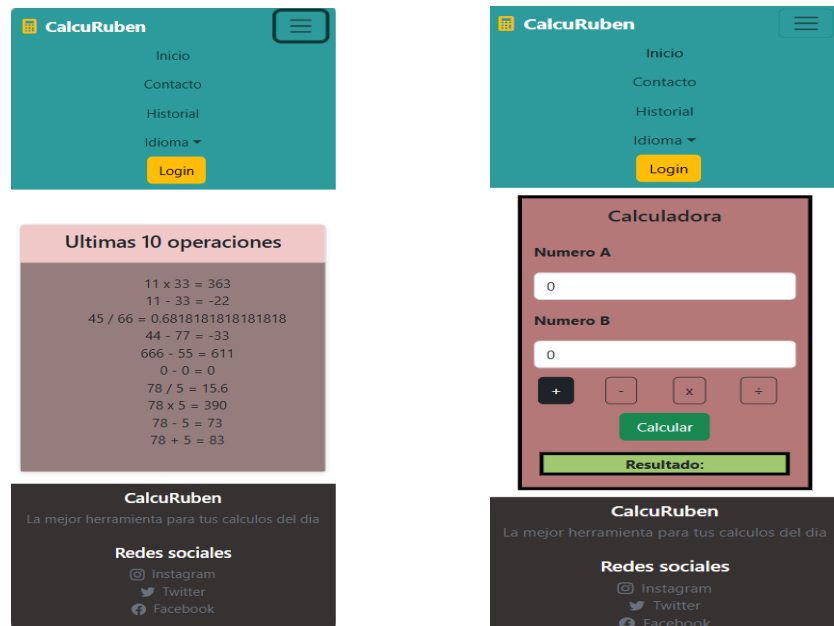


Fig 8. Algunas capturas de como se ve el móvil

3.1. Footer.jsx

El componente de pie de página se ha estructurado utilizando un sistema de rejilla. La disposición de los elementos se adapta dinámicamente según el ancho de la pantalla del usuario.

Para lograr este comportamiento, se ha configurado la división de las columnas de la siguiente manera:

- **xs={12}**: En resoluciones pequeñas (móviles), cada columna ocupa el 100% del ancho del contenedor (las 12 columnas del grid). Esto permite que los bloques de información se apilen verticalmente, evitando el desbordamiento horizontal y facilitando la lectura.
- **md={4}**: En resoluciones medias y superiores (tablets y ordenadores), el espacio se redistribuye para que cada bloque ocupe solo un tercio del ancho total (4 de las 12 columnas).

3.2. Calculadora.jsx

Para que la calculadora se vea bien tanto en el móvil como en el ordenador, he tratado con los tamaños de las columnas (xs y sm):

- Se ha asignado un valor de 12 a las columnas de los campos de entrada (Número A y Número B). Esto garantiza que en dispositivos móviles (pantallas extra pequeñas), cada elemento ocupe el 100% del ancho disponible.

- A diferencia de los inputs, los botones de operación utilizan un valor de 3. Esto permite que, incluso en móviles, los cuatro botones queden alineados en una sola fila ($3 \times 4 = 12$), ahorrando espacio vertical y manteniendo las herramientas de cálculo siempre a la vista.
- Al superar el breakpoint de pantallas pequeñas (sm), las columnas reducen su ancho relativo. Esto hace que las etiquetas y los inputs dejen de estar uno debajo del otro para pasar a una disposición horizontal.

3.3. Header

La barra de navegación es uno de los elementos que más cambia para adaptarse a las pantallas pequeñas:

- Al usar `<Navbar expand="lg">`, le estoy diciendo a la web que en pantallas grandes muestre todos los enlaces a la vista, pero que en pantallas más pequeñas (móviles y tablets) los esconda automáticamente.
- He incluido el `<Navbar.Toggle />`, que es el componente que hace aparecer el típico botón de tres rayas en el móvil. Al pulsarlo, se despliega el contenido que está dentro del `<Navbar.Collapse>`, permitiendo que el usuario vea el Inicio, Contacto, Historial y el selector de idiomas de forma ordenada hacia abajo.

4. Conceptos adicionales

4.1. Control mediante historial

Aunque no era un requisito, he querido implementar un sistema de historial para que el usuario pueda llevar un registro de los cálculos que ha ido haciendo. Esto me ha servido para practicar cómo guardar información en una lista (un array de objetos) y cómo pasar esos datos entre diferentes componentes de la aplicación.

4.2. Campos adicionales

Además de la calculadora, he añadido secciones como la página de contacto o el login. Como he mencionado antes, no tienen una funcionalidad real detrás, pero me han servido para completar la estructura de la web y que no fuera solo una página vacía, practicando así la maquetación y el uso de formularios.

4.3. Idiomas

He implementado un sistema para que la aplicación se pueda usar tanto en castellano como en inglés. Sé que en proyectos reales esto se suele hacer con librerías específicas, ya que meter todo el texto a mano en un objeto puede ser mucho lío si la web crece.

Sin embargo, he decidido hacerlo así porque me ha venido muy bien para practicar a fondo cómo enviar las props de un lado a otro y cómo gestionar un estado global que afecte a toda la aplicación al mismo tiempo.